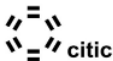


CURSO EXPERTO EN IA Y ALGORITMOS VERDES MICROCREDENCIAL ALGORITMOS VERDES PARA LA IA: DISEÑO E IMPLEMENTACIÓN

Efficiency in Computer Vision

Monday, October 13th, 2025

Noelia Barreira Rodríguez



UNIVERSIDADE DA CORUÑA



Financiado por la
Unión Europea
NextGenerationEU



GOBIERNO
DE ESPAÑA

MINISTERIO
PARA LA TRANSFORMACIÓN DIGITAL
Y DE LA FUNCIÓN PÚBLICA



29
26

COMPUTER VISION

Goal Interpret, analyze and understand information from images and videos

- Tasks**
- Image classification
 - Object detection
 - Segmentation
 - Object tracking
 - Pose estimation
 - Image generation
 - ...

CURRENT TRENDS IN COMPUTER VISION

Task	Dominant approaches
Image classification	CNN ¹ , ViT ²
Object detection	CNN, ViT
Segmentation	CNN, ViT
Object tracking	CNN, ViT
Pose estimation	CNN, ViT
Image generation	Diffusion models, GANs ³

¹Convolutional Neural Networks

²Vision Transformers

³Generative Adversarial Networks

CURRENT TRENDS IN COMPUTER VISION

Task	Dominant approaches
Image classification	CNN ¹ , ViT ²
Object detection	CNN, ViT
Segmentation	CNN, ViT
Object tracking	CNN, ViT
Pose estimation	CNN, ViT
Image generation	Diffusion models, GANs ³

 Deep learning models!

¹Convolutional Neural Networks

²Vision Transformers

³Generative Adversarial Networks

DRAWBACKS OF DEEP LEARNING APPROACHES IN COMPUTER VISION

- High energy consumption for training
- Inefficient inference
- Preprocessing and storage of massive datasets
- Expensive hyperparameter tuning and architecture design
- Hardware requirements

GREEN AI SOLUTIONS

- Encourage model reuse and sharing
 - Transfer learning
- Design efficient architectures suitable for any device
- Use foundation models
- Reduce model size and complexity
 - Pruning
 - Quantization
 - Distillation
- Find optimal architectures
 - Efficient Neural Architecture Search

Part I

Deep Learning models for Computer Vision

IMAGE CLASSIFICATION PIPELINE

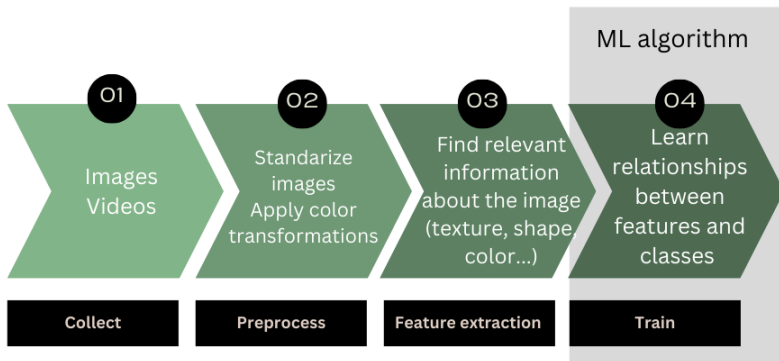
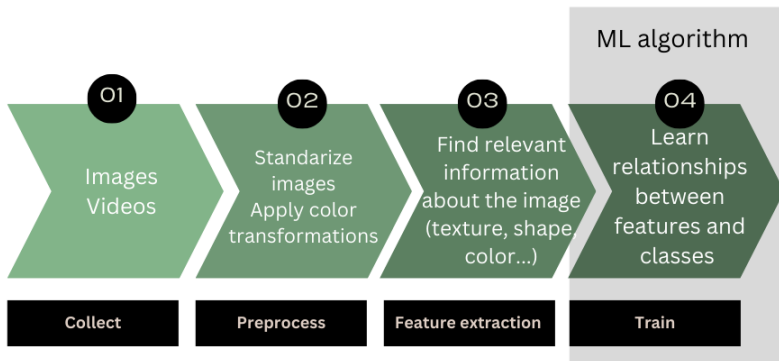


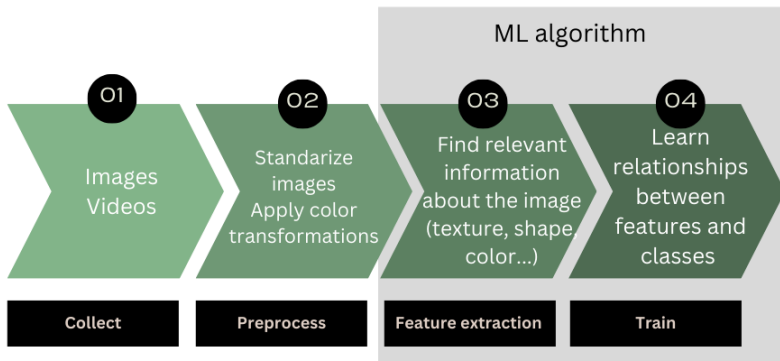
IMAGE CLASSIFICATION PIPELINE



Warning

Extraction of features and classification are decoupled

IDEAL IMAGE CLASSIFICATION PIPELINE



IMAGES

- Grid of pixel values that represent visual information
- Each pixel encodes color and/or intensity data
- Typically pixel values range between 0-255 or 0-1

255	225	195	165
225	195	165	135
195	165	135	105
165	135	105	75

Grayscale Image (4x4 pixels)

IMAGES

- Grid of pixel values that represent visual information
- Each pixel encodes color and/or intensity data
- Typically pixel values range between 0-255 or 0-1

Red Channel

50	100	150	200
50	100	150	200
50	100	150	200
50	100	150	200

Green Channel

50	50	50	50
100	100	100	100
150	150	150	150
200	200	200	200

Blue Channel

255	225	195	165
225	195	165	135
195	165	135	105
165	135	105	75

Composite RGB Image

50,50,255	100,50,225	150,50,195	200,50,165
50,100,225	100,100,195	150,100,165	200,100,135
50,150,195	100,150,165	150,150,135	200,150,105
50,200,165	100,200,135	150,200,105	200,200,75

FILTERS

In classic Computer Vision, **filters** are used to identify patterns

- Edges
- Textures
- Shapes

e.g. this input image convolved with this filter outputs



-1	-1	-1
2	2	2
-1	-1	-1



FILTERS

In classic Computer Vision, **filters** are used to identify patterns

- Edges
- Textures
- Shapes

e.g. this input image convolved with this filter outputs



-1	-1	-1
2	2	2
-1	-1	-1

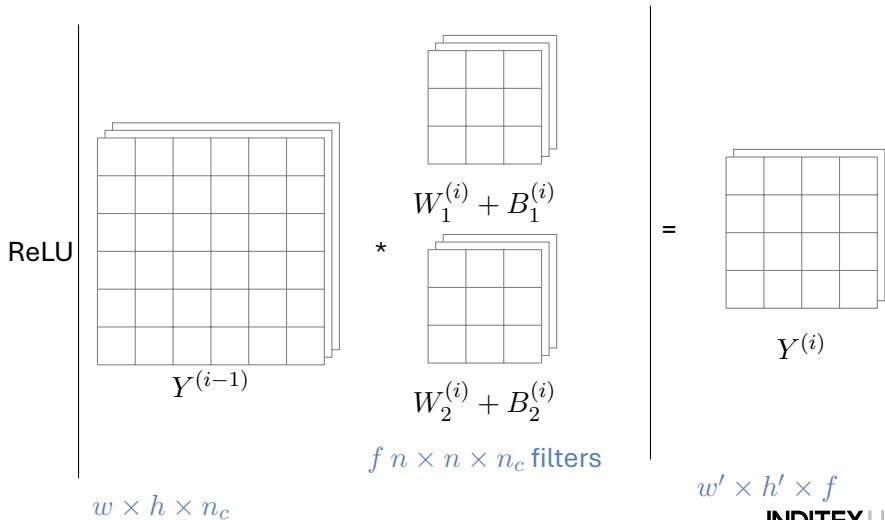


Idea

Include filters in the network to extract features

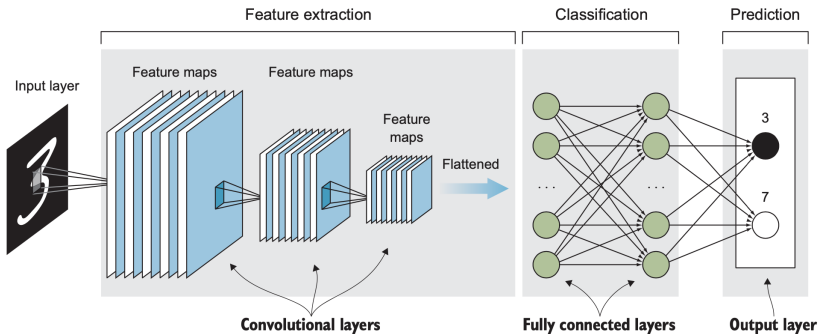
- In the training stage, learn filter coefficients to extract best features

CONVOLUTIONAL LAYER



CONVOLUTIONAL NEURAL NETWORK

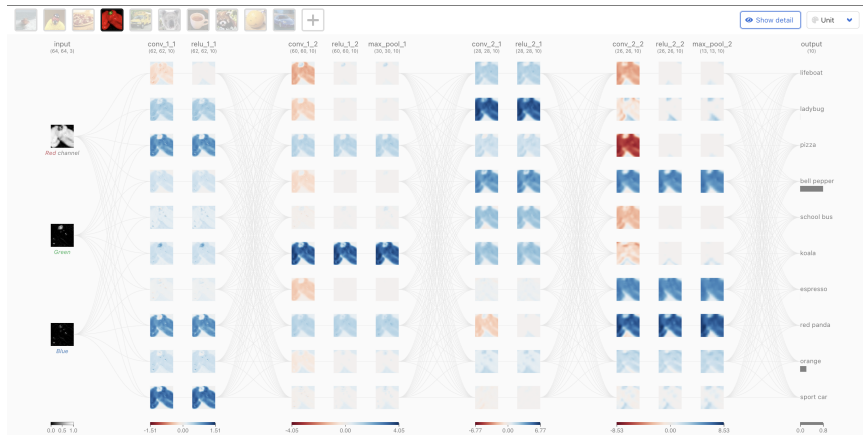
- The output of a convolutional layer is a feature map
- Width and height of feature maps decreases as we go deeper
 - Convolutions with stride
 - Pooling layers



- Softmax activation function in the output layer

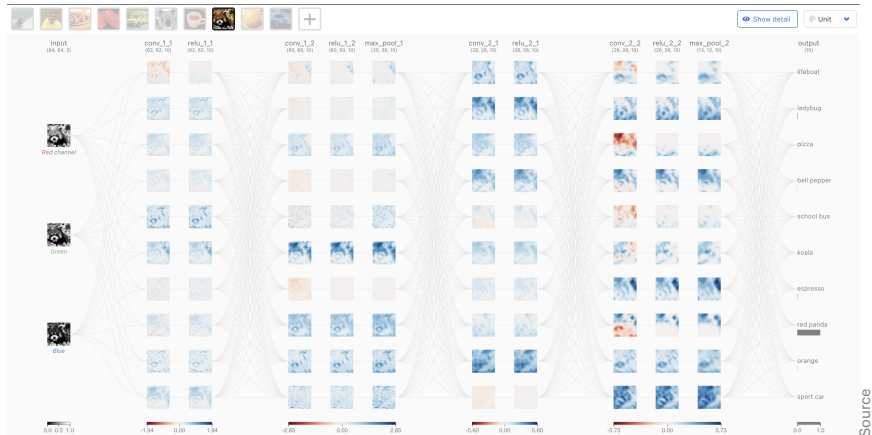
CONVOLUTIONAL NEURAL NETWORK

What happens in inference?



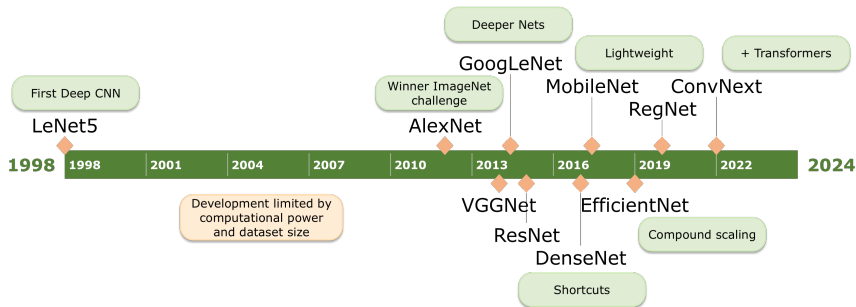
CONVOLUTIONAL NEURAL NETWORK

What happens in inference?



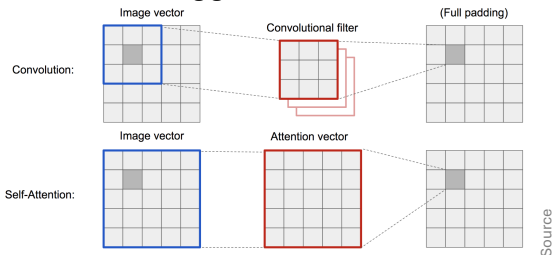
CONVOLUTIONAL NEURAL NETWORKS

History



FROM CNNs TO TRANSFORMERS

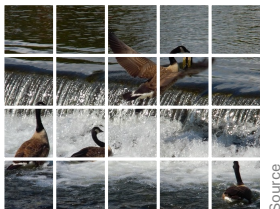
- CNNs exploit local patterns and use convolution to extract spatial features
- CNNs struggle with
 - Capturing long-range dependencies
 - Scaling up without loss of resolution or receptive field
- Transformer architecture was adapted to image data
 - Good at modeling global context via self-attention



VISION TRANSFORMERS (ViT)

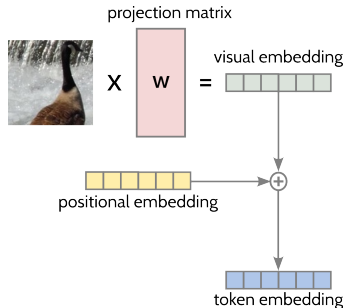
💡 Treat an image as a **sequence of patches** (words)

1. Image is divided into fixed-size non overlapping patches



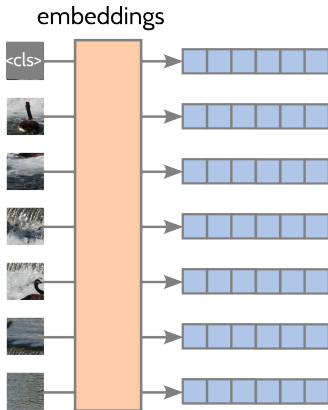
2. Each patch is flattened and projected to create a visual embedding

3. A positional embedding is added to retain spatial order



VIT EMBEDDING PATCHES

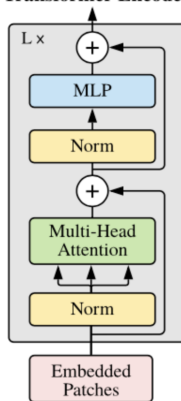
- A special [CLS] token is prepended for classification tasks



VIT ARCHITECTURE

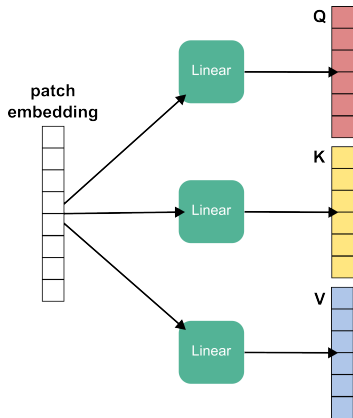
- Uses only the **transformer encoder**
- L layers that consists of
 - Multi-Head Self-Attention
 - Feed-Forward Network
 - LayerNorm and residual connections

Transformer Encoder



Source

QUERIES, KEYS AND VALUES

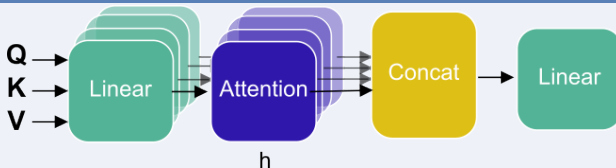


For each patch embedding

- Apply three independent linear transformations
 - Query
 - Key
 - Value
- Use these projections to perform comparisons

MULTI-HEAD SELF-ATTENTION

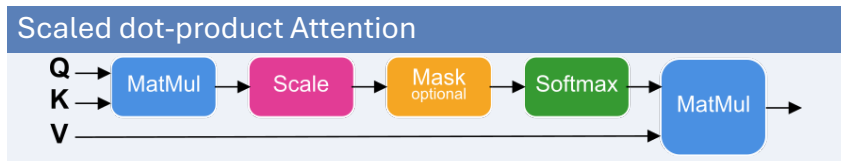
Multi-head Attention



1. Project the input queries, keys and values to h different linear spaces
2. For each projected set of vectors (Q, K,V)
 - 2.1 Compute the scaled dot-product attention
3. Concatenate the output vectors of the attention modules
4. Perform a linear projection

💡 Each attention head will learn to focus on specific aspects of similarity between the embeddings

SELF-ATTENTION HEAD



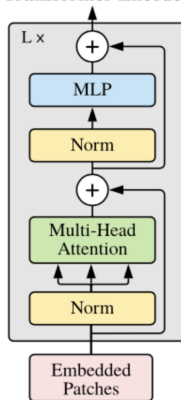
For each query (patch embedding)

1. Compute the similarity **scaled dot-product + softmax** between
 - Query **current patch embedding**
 - Keys **other patch embeddings**
2. Create a new vector as a combination **dot-product**
 - Similarity weights
 - Values **patch embeddings**

MULTILAYER PERCEPTRON

- Process tokens independently
- Extract patterns from the attention vectors

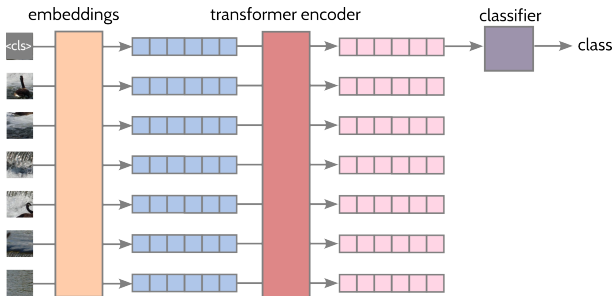
Transformer Encoder



Source

CLASSIFICATION HEAD

- The <CLS> token represents the entire image
- The output vector corresponding to this token is the input of a classifier
 - Typically, a Multilayer Perceptron



👁 Remaining output vectors can be used for other tasks

Part II

Transfer learning

TRANSFER LEARNING [1]

💡 If a model is trained on a large and general enough dataset, this model could serve as a generic representation of the visual world

1. Download a trained open source model
 2. Use their optimized parameters as starting point to train a new model on a smaller dataset for a given problem
- ➔ Train the model faster and achieve better results

TRANSFER LEARNING

Problems addressed

Data Training a network from scratch requires a lot of data

- It is rare to have a large dataset to solve a problem
- It is expensive acquire and label data

Computation Training a model from scratch usually requires days or weeks

Other benefits

- Generalization
- Avoid overfitting

HOW TRANSFER LEARNING WORKS

What is learned by the network during training?

How to compute suitable feature maps from images

How is that learned?

By backpropagation, weights are updated to minimize the error function

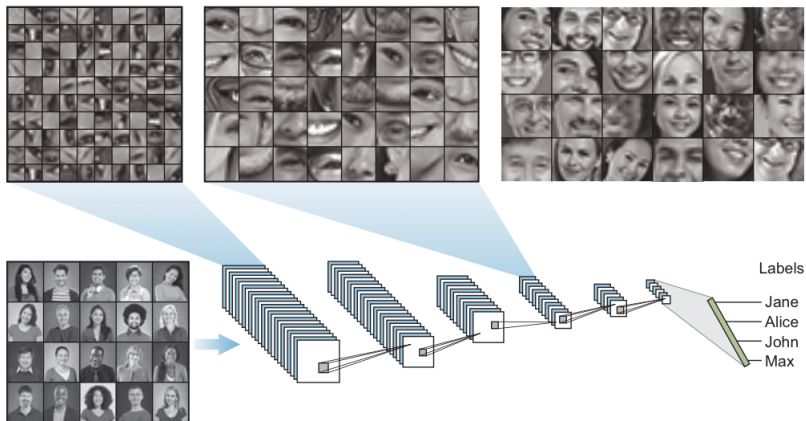
What is the relationship between weights and features?

A feature map is the result of applying the weight filter on the input image by means of the convolution process

What is really being transferred?

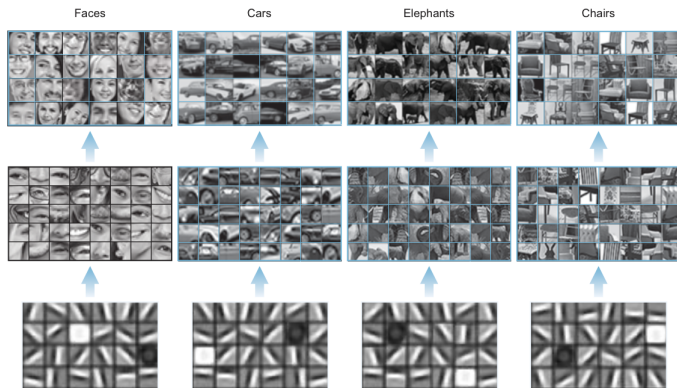
The optimized weights of the pretrained network

HOW DO NEURAL NETWORKS LEARN FEATURES?



The more deeper you go through the network layers, the more image-specific features are learned

HOW DO NEURAL NETWORKS LEARN FEATURES?



Low level features at early stages are similar in different domains

TRANSFER LEARNING APPROACHES

Use the pretrained network directly

1. Download a pretrained network
2. Use as is for the target task

 It only works if the source domain and the target domain are similar

Example

- We have to identify images of dogs
- ImageNet has labeled images for 15 breeds of dogs
- ➔ Use a pretrained network on ImageNet for this task

TRANSFER LEARNING APPROACHES

Frozen backbone

1. Download a pretrained network
2. Freeze the feature extraction part
3. Remove the clasification/segmentation/detection head
4. Train a new classification/segmentation/detection head

Example

- We have to identify only images of dogs and cats (2 classes)
- ImageNet has labeled 1000 different classes
- We have a small dataset of images with dogs and cats
- ➔ Freeze the feature extraction part of a pretrained network on ImageNet and train the last fully connected layers with the dataset

TRANSFER LEARNING APPROACHES

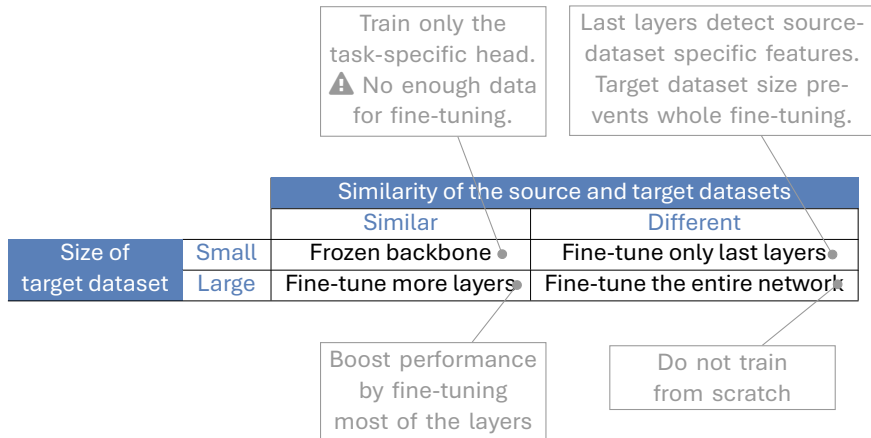
Fine-tuning

💡 Useful when the target domain is not similar to the source domain

1. Download a pretrained network
2. Freeze some layers only (or none)
 Different domains Freeze only the first layers
 Similar domains Freeze more layers
3. Train the network

	Training from scratch	Fine-tuning
Weight initialization	Random	Optimized weights in source domain
Convergence	Slower	Faster

SCENARIOS FOR TRANSFER LEARNING



OPEN SOURCE DATASETS

Name	Description	Size	Class.	Detec.	Segm.
ImageNet	Standard for image classification	1.2M	✓	✓	
CIFAR-10/100	32 × 32 images	60K	✓		
MNIST	28 × 28 handwritten digits	60K	✓		
Fashion-MNIST	28 × 28 fashion images	60K	✓		
COCO	Common objects in context	330K		✓	✓
Pascal VOC	Early benchmark	20K	✓	✓	✓
Open Images	Diverse web images	9M	✓	✓	✓
Cityscapes	Urban street scenes	20K		✓	✓
Mapillary Vistas	Traffic scenes	25K		✓	✓
KITTI	Traffic scenes (camera, GPS, lidar)	15k		✓	✓

More datasets <https://www.kaggle.com/>

Models and datasets <https://huggingface.co/>

Models <https://pytorch.org/hub/>

Part III

Efficient Architectures for Computer Vision

GENERAL PURPOSE ARCHITECTURE

Backbone network Extracts deep representations from raw inputs

- Images
- Video frames
- Point clouds

Task-specific head Designed for the task of interest

COMPUTER VISION TASKS

- Image classification
- Object detection
- Segmentation
 - Instance segmentation
 - Semantic segmentation
 - Panoptic segmentation
- Image generation

Section 1

Image classification

EFFICIENT ARCHITECTURES FOR IMAGE CLASSIFICATION

CNN-based

- SqueezeNet (2016)
- MobileNet (2017)
- EfficientNet (2019)

ViT-based

- MobileViT (2021)

SQUEEZENET [2]

Architectural design strategies

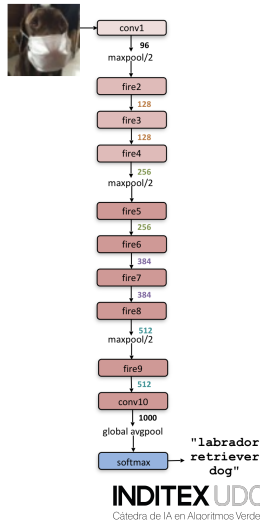
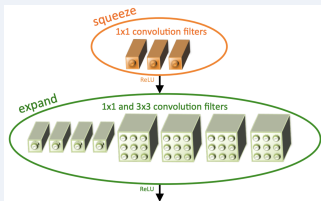
Lightweight CNN designed for efficiency

Fire module

Squeeze layer 1×1 filters

- Reduces the number of parameters
- Limits the number of input channels to the 3×3 filters

Expand layer Mix of 1×1 and 3×3 filters



SQUEEZENET [2]

Architectural design strategies

Global Average Pooling

- Replace fully connected layers
- Computes the average value of each channel

Example

- If the last layer outputs a $7 \times 7 \times 1024$ matrix, GAP will
 - Compute the average of each 7×7 channel
 - Produce a 1024-dimensional vector
- This tensor is the input of a softmax layer (or other classifier)
- An alternative architecture with a single fully connected layer will have $7 \times 7 \times 1024 \times n_{classes} = 50176 \times n_{classes}$ parameters

SQUEEZENET [2]

Results on ImageNet

Architecture	Compression	Quantization	Size vs AlexNet
AlexNet	None	32 bits	1x
AlexNet	Deep Compression	5-8 bits	35x
SqueezeNet	None	32 bits	50x
SqueezeNet	Deep Compression	8 bits	363x
SqueezeNet	Deep Compression	6 bits	510x

80,3% Top-5 ImageNet Accuracy in all the models

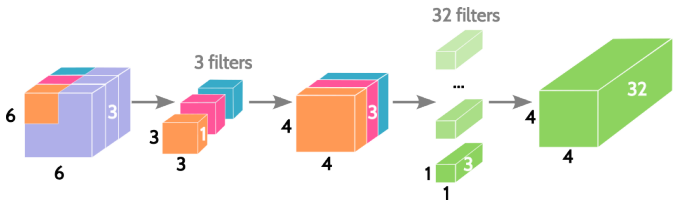
MOBILENET [4]

Architectural design strategies

- Depthwise separable convolutions

Depthwise convolution Single filter per input channel

Pointwise convolution (1×1) Combines the output of the depthwise step across channels



- Reduces the number of parameters (and computational cost!)
 - $3 \times 3 \times 32 = 288$ vs $3 \times 3 \times 3 + 1 \times 1 \times 3 \times 32 = 123$

MOBILENET [4]

Architectural design strategies

- Width (α) multiplier
 - Decreases the number of filters in each layer according to a factor (model width)
 - Allows trade-off between accuracy and efficiency
- Resolution (ρ) multiplier
 - Downscales the input image resolution
 - Reduces computational costs

MOBILENET [4]

Example of resource usage

$14 \times 14 \times 512$ input feature map

$512 \ 3 \times 3$ filters

Optimization	MFLOPs ⁴	Parameters
Standard convolution	462	2.36M
Depthwise Separable Convolutions	52.3	0.27M
$\alpha = 0.75$	29.6	0.15M
$\rho = 0.714$	15.1	0.15M

⁴Million Floating Point Operations per second

MOBILENET [4]

Results on ImageNet

Model	Accuracy	MFLOPs	Parameters
MobileNet-224 ⁵	70.6%	569	4.2M
GoogleNet	69.8%	1550	6.8M
VGG 16	71.5%	15300	138M
MobileNet-160 ⁶	60.2%	76	1.32M
Squeezenet	57.5%	1700	1.25M
AlexNet	57.2%	720	60M

⁵224 × 224 image resolution

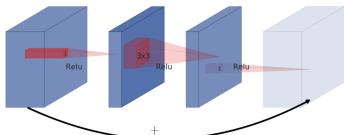
⁶160 × 160 image resolution

MOBILENET V2 [5]

Architectural design strategies

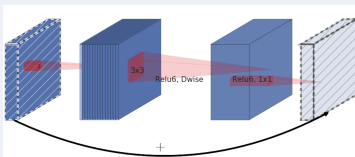
Residual block

1. Compress # channels with a 1×1 convolution
2. Apply a 3×3 convolution
3. Increase # channels with a 1×1 convolution
4. Add the input to the output



Inverted residual block

1. Widen # channels with a 1×1 convolution
2. Use a 3×3 depthwise convolution
3. Reduce # channels with a 1×1 convolution
4. Add the input to the output



MOBILENET V2 [5]

Results on ImageNet

Model	Accuracy	MFLOPs	Parameters	Pixel 1 time
MobileNet v1	70.6%	575	4.2M	113ms
MobileNet v2	72.0%	300	3.4M	75ms
MobileNet v2 ⁷	74.7%	585	6.9M	143ms

⁷ $\alpha = 1.4$

MOBILENET V3 [6]

Architectural design strategies

- NAS to optimize network architecture
- Redesign expensive layers (e.g. drop expensive layers at no loss of accuracy)
- Hard swish activation function

$$hswish(x) = x \frac{ReLU6(x + 3)}{6}$$

- Replaces the swish activation function that improves the accuracy

$$swish = x\sigma(x)$$

- Sigmoid function is expensive to compute on mobile devices

MOBILENET V3 [6]

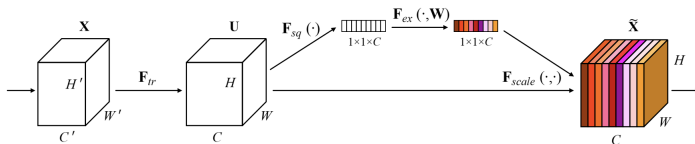
Architectural design strategies

- Squeeze-and-Excitation (SE) block
 1. Apply Global Average Pooling to each channel
 2. Pass this vector to a two-layer MLP

$$s = \sigma(W_2 \cdot ReLU(W_1 \cdot z))$$

Each value in this vector represents a channel attention weight

3. Multiply the original input by the channel attention weights
 - This emphasizes important channels and suppresses less relevant ones



MOBILENET V3 [6]

Results on ImageNet

Model	Accuracy	MFLOPs	Parameters	CPU time
V2	72.0%	300	3.4M	64ms
V3 - Large ⁸	75.2%	219	5.4M	51ms
V3 - Large ⁹	73.3%	155	5.0M	39ms
V3 - Small ¹⁰	67.4%	56	2.5M	15.8ms
V3 - Small ¹¹	65.4%	44	2.0M	12.8ms

⁸ $\alpha = 1.0$

⁹ $\alpha = 0.75$

¹⁰ $\alpha = 1.0$

¹¹ $\alpha = 0.75$

EFFICIENTNET [8]

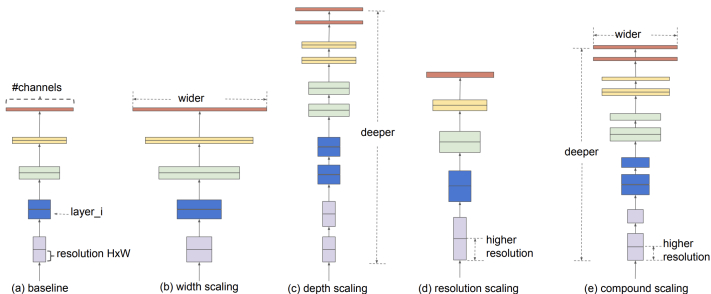
Architectural design strategies

- Compound model scaling

⚠ Scaling up width, depth or resolution improves accuracy, but the accuracy gain diminishes for bigger models

💡 Balance all dimensions during scaling

$$\text{depth} = \alpha^\phi \quad \text{width} = \beta^\phi \quad \text{res.} = \gamma^\phi \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad \alpha, \beta, \gamma \geq 1$$



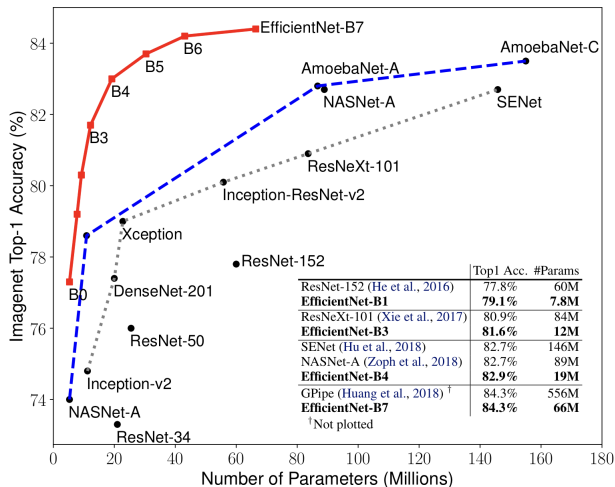
EFFICIENTNET [8]

Architectural design strategies

- EfficientNet-B0, an efficient baseline model
 - Inverted bottleneck blocks (MobileNet V2)
 - Squeeze-and-Excitation (SE) blocks (MobileNet V3)
 - SiLU (swish) activation function
 - Neural Architecture Search for optimal
 - Kernel sizes
 - Expansion ratios
 - Repetition counts
- Efficient-B1 to B7 are obtained by scaling up the baseline network

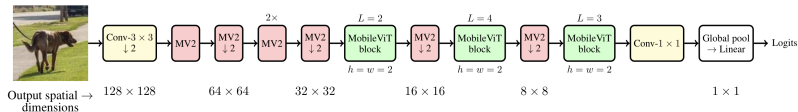
EFFICIENTNET [8]

Results on ImageNet



MOBILEVIT [9]

Hybrid CNN + Transformer architecture



MobileNetV2 blocks

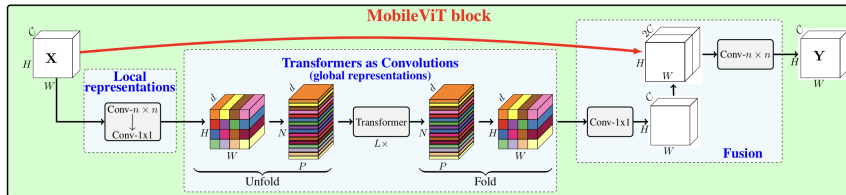
- Capture local spatial information efficiently
- Used for early local processing

MobileViT blocks

- Processes small patches
- Less computation than full-image attention

MOBILEViT [9]

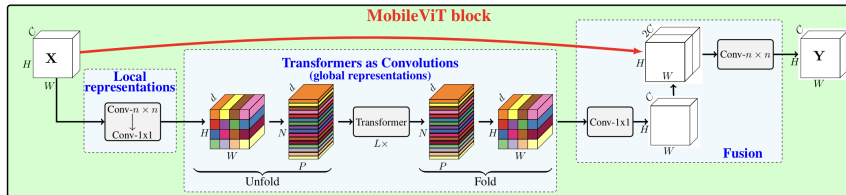
MobileViT blocks



1. Use a **convolutional layer** to encode spatial information
2. Apply a **pointwise convolutional layer** to project the tensor to a high-dimensional space
3. Unfold the tensor into **flattened patches**
4. Compute the inter-patch relationships via a **transformer**

MOBILEViT [9]

MobileViT blocks



5. Fold the resulting tensor
6. Apply a **pointwise convolution**
7. Concatenate with the input tensor
8. Apply a convolutional layer

- ⚙️ Convolution $\approx$ unfolding, matrix multiplication and folding
- 💡 Replace local processing (matrix multiplication) by global processing (transformer)

MOBILEViT [9]

Results on ImageNet-1k

Model	# Params	Top-1 Accuracy
MobileNetv2	2.6M	68.4
ShuffleNetv2	2.3M	69.4
MobileViT-XS	2.3M	74.8

Model	# Params	Top-1 Accuracy
EfficientNet-B0	5.3M	76.3
ResNet-101	44.5M	77.4
MobileViT-S	5.6M	78.4

Section 2

Object detection

EFFICIENT ARCHITECTURES FOR OBJECT DETECTION

Object detection

Computer vision technique used to identify and locate objects within images or videos

- Autonomous driving
- Surveillance

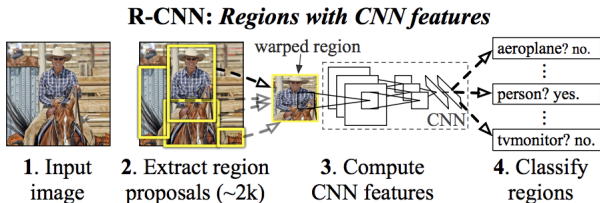
Two-stage detectors Region proposal + classification and refinement

- R-CNN family

One-stage detectors Object location and classification at the same time

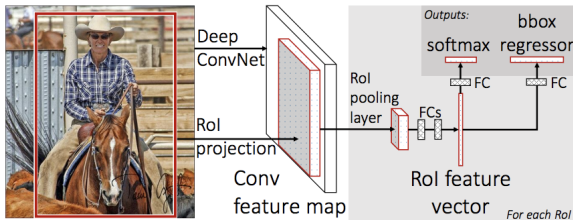
- YOLO
- EfficientDet
- DETR

R-CNN [10]



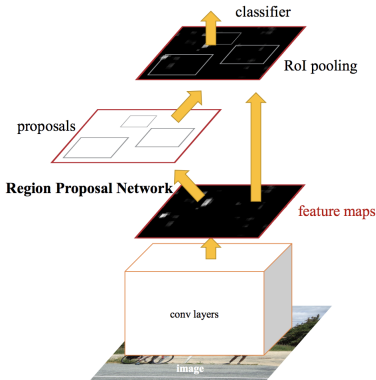
- ⚠ Selective Search Algorithm finds candidate regions
- ⚠ Each warped region is passed through a CNN to compute features
- ⚠ An independent SVM classifies the features in each region
- ⚠ A bounding box regressor adjust the location of the regions

FAST R-CNN [11]



- ⚠ Selective Search Algorithm finds candidate regions
- ➔ The image is passed once through an **efficient** CNN to get the features
- ➔ The classification and bounding box regressor are integrated in the network

FASTER R-CNN [12]



- The image is passed once through an **efficient** CNN to get the features
- Region proposal network is integrated in the network to find candidate regions
- The classification and bounding box regressor are integrated in the network

R-CNN FAMILY

Results on Pascal VOC 2007 dataset¹²

R-CNN

- ≈ 50 seconds per image
- 66.0 mAP

Fast R-CNN

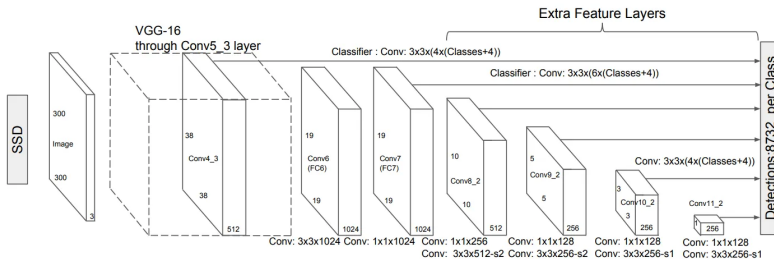
- ≈ 2 seconds per image
- 66.9 mAP

Faster R-CNN

- ≈ 0.2 seconds per image
- 66.9 mAP

¹²As published

SINGLE SHOT DETECTORS (SSD) [13]



- Compute features using a CNN
- Detect objects at different scales
 - For each position at the feature map output n bounding boxes
 - (x, y, w, h) of the bounding box
 - Probabilities of each class

SINGLE SHOT DETECTORS

Results on PASCAL VOC2007 dataset

Model	mAP	FPS	# Boxes	Image resolution
Faster R-CNN	73.2	7	≈ 6000	1000×600
SSD300	74.3	46	8732	300×300
SSD512	76.8	19	24564	512×512

YOU ONLY LOOK ONCE (YOLO)

Single-stage detection

- No region proposal steps
- It predicts bounding boxes and class probabilities at the same time
- ➔ Faster than two-stage detectors

Fully convolutional design

- It does not contain fully connected layers
- From v2 onward
- ➔ Speed and scalability improved

YOU ONLY LOOK ONCE (YOLO)

Grid-based prediction and few proposals per grid cell

- Image is divided into a $S \times S$ grid
- Each grid cell is responsible for predicting objects whose centers fall inside it
- Only a small fixed number of bounding boxes is predicted per grid cell
- Number of operations is reduced

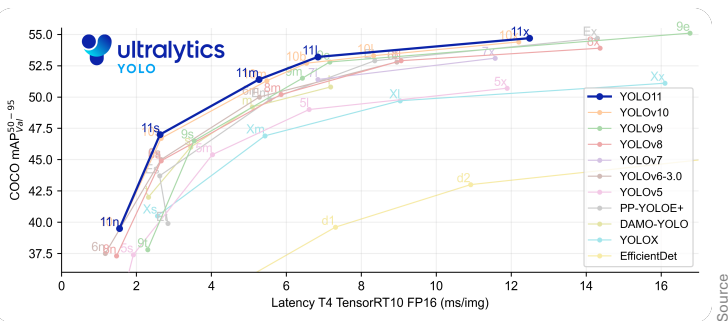
Lightweight variants

- Later versions create a family of YOLO networks
- Tiny and nano versions appropriate for edge devices and mobile CPUs

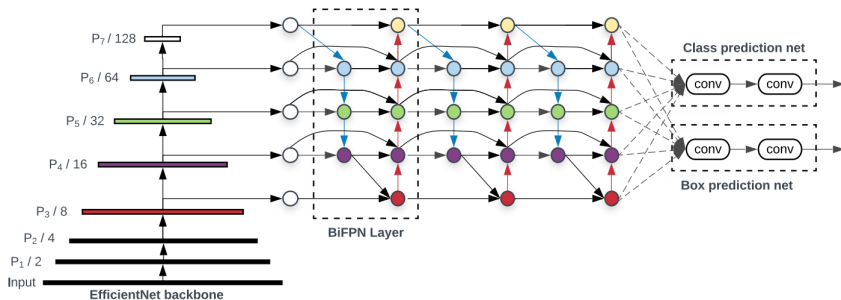
YOLO

Results on COCO dataset

Fast, but not so accurate



EFFICIENTDET [20]



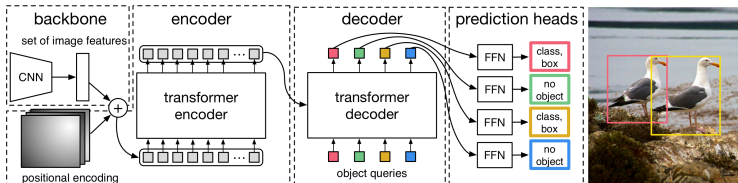
- EfficientNet backbone
- Bidirectional Feature Pyramid Network (BiFPN)
 - Addresses the multi-scale feature fusion problem
 - Fuses top-down and bottom-up features

EFFICIENTDET

Results on COCO dataset

Model	FLOPS	AP_{50}	Latency on V100 (ms)
YOLOv3	71B	57.9	-
EfficientDet-D0	6.1B	53.0	10.2
EfficientDet-D3	25B	65.9	29.0
EfficientDet-D7	325B	72.5	122

DETR [21]



1. Extract details from images using a CNN
 2. Obtain a rich set of image representations via the transformer encoder
 3. Send n different queries to look at the image representation and get the answer through the transformer decoder
 - Object queries are embeddings learned during training
 - Each object query focus on particular spatial patterns
 4. Output the bounding box and predict the class per object query (or no object) (FFN)
 - Performance similar to Faster R-CNN
- ⚠ Slow convergence and high computational cost

EFFICIENT DETR [22]

- Uses only 3 encoder layers and 1 decoder layer (6 encoders and 6 decoders in DETR)
 - Uses only 100 queries like DETR (less than other DETR-based models)
 - Two-stage detector
 - A detection head predicts coarse region proposals
 - Transformer-based refinement of top-K proposals
 - Both detectors share the same detection head
- ➔ Better accuracy, lower computational cost and fewer parameters

EFFICIENT DETR [22]

Results on COCO dataset

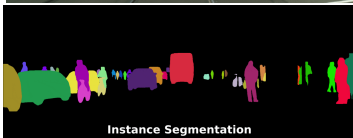
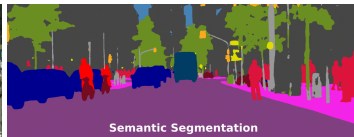
Model	Epochs	# Params	AP_{50}
Faster R-CNN-FPN-R50	36	42M	61.0
DETR-R50	500	41M	62.4
Efficient DETR-R50	36	32M	62.2

Section 3

Segmentation

IMAGE SEGMENTATION

- Assign a class label to each image pixel
- Types



Source

EFFICIENT ARCHITECTURES FOR SEMANTIC SEGMENTATION

Encoder-Decoder architectures with efficient backbones

- U-Net
- DeepLab family

Transformers and hybrid models

- MobileViT + DeepLabv3 segmentation head

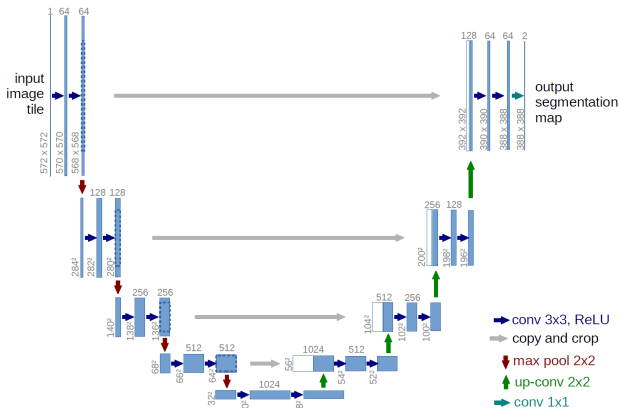
Lightweight architectures

- BiSeNet
- PidNet

ENCODER-DECODER ARCHITECTURES WITH EFFICIENT BACKBONES

UNet [23]

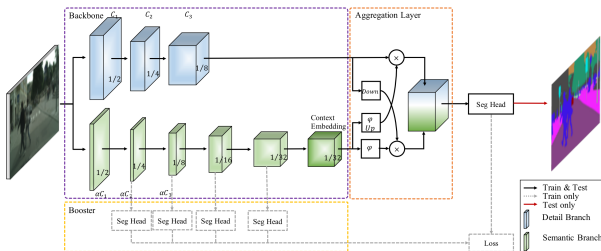
Replace the encoder with an efficient backbone
(MobileNet/EfficientNet)



Variants: Mobile U-Net uses depth-wise separable convolutions

BILATERAL SEGMENTATION NETWORK (BISENET) [24]

Real-time semantic segmentation



Bilateral segmentation backbone

Detail branch captures spatial details

Semantic branch focused on semantics

➔ Parallelizes low and high frequency feature extraction

BISENET [24]

Gather-and-Expansion Layer

Applied in the Semantic branch

1. 3×3 convolution to aggregate feature responses and expand to a higher dimensional space
2. 3×3 depth-wise convolution over each individual output channel
3. 1×1 convolution to project the output into a low channel capacity space

→ Lower FLOPs

Aggregation Module

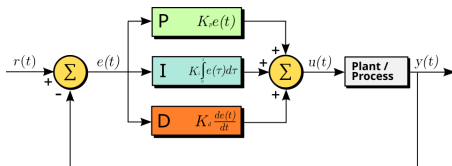
- Fuse the information of both branches

→ Sharpens boundaries with minimal overhead

PIDNET [25]

PID controller

Feedback-based control loop mechanism used to manage machines and processes that require continuous control and automatic adjustment



Proportional (P) reacts to current error

➔ If the error is large, the response is stronger

Integral (I) accumulates past error over time

➔ Eliminates bias

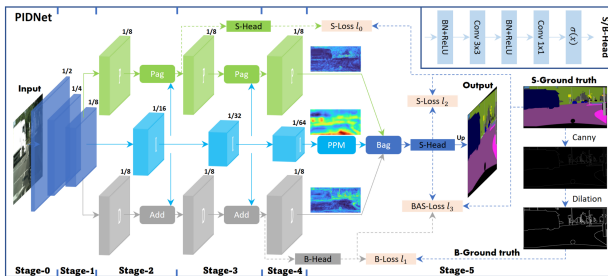
Derivative (D) predicts future error change

➔ Improves stability

💡 Use this basic idea in semantic segmentation

PIDNET [25]

- Three branches
 - **P** captures fine-grained spatial details
 - **I** captures global semantic context
 - **D** stabilizes and refines the output
- P and I branches process input in parallel
- Each branch operates at different resolutions
- D uses attention-like mechanisms to balance features from the other branches and fuse data



REAL TIME SEMANTIC SEGMENTATION

Results on Cityscapes

Model	# Params	GFLOPS	Accuracy	FPS ¹³
BiSeNet v1	13.3M	15	74.8	105
BiSeNet v2-L	3.5M	12	75.8	47.3
PIDNet-S	7.6M	47.6	78.6	93.2
PIDNet-L	36.9M	2276	80.9	31.1

¹³Computed with different hardware configurations

EFFICIENT ARCHITECTURES FOR INSTANCE SEGMENTATION

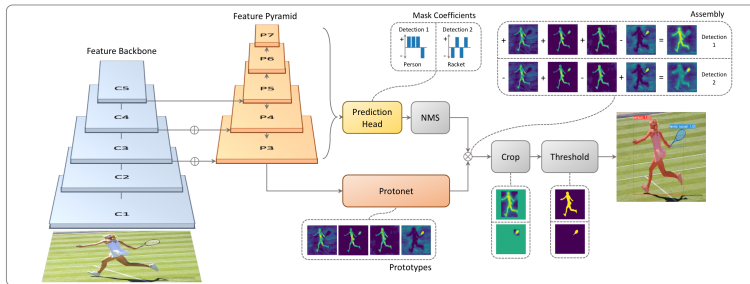
Detection models with segmentation heads Predict a mask for each bounding box detected

- EfficientDet + segmentation head
- Mask R-CNN (Faster R-CNN + segmentation head)

Lightweight architectures

- YOLACT
- SOLO

YOU ONLY LOOK AT COEFFICIENTS (YOLACT) [26]



Mask generation

- Generates a small set of global prototype masks
- Each object is assigned linear combination coefficients to produce its instance mask from these prototypes
- ➔ Speedup in mask generation, especially when detecting many objects

YOLOACT [26]

One-stage detection backbone

- No region proposal stage
- End-to-end inference in a single forward pass
- ➔ Faster than two stage detectors

Parallel mask and box prediction

- ➔ Reduction in total inference time

Modular and scalable design

- Easy scaling with lighter backbones
- The number of prototype masks and coefficients can be adjusted

YOLACT [26]

Results on COCO dataset

Model	mAP	FPS
YOLACT	29.8	33.5
Mask R-CNN	37.5	5
YOLACT++	34.1	33

Part IV

Foundation models

ZERO-SHOT IMAGE CLASSIFICATION

Traditional Image Classification

- Large, labeled datasets are needed
- Models struggle to generalize when the number of examples is small
- Models are limited in their ability to classify unseen data

Zero-Shot Image Classification

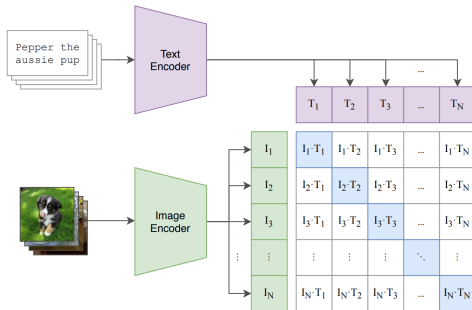
- The model sees zero labeled examples of target classes during training
- It uses descriptive information to make predictions
 - Text
 - Attributes
 - Embeddings

ZERO-SHOT IMAGE CLASSIFICATION

Contrastive Language-Image Pre-Training (CLIP) [27]

Multi-modal model

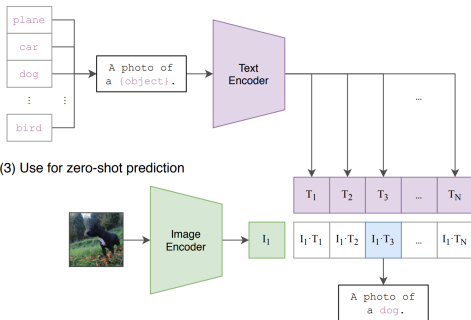
- Training dataset consists of photo-description pairs



- Text encoder (Transformer)
- Image encoder (ViT/ResNet)
- Both embeddings are represented in the same space
- Contrastive Loss for training
 - Correct/incorrect pairs

CLIP [27]

(2) Create dataset classifier from label text



1. Encode classes

- Create textual descriptions

2. Encode the image

3. Predict the most similar class

- Cosine similarity

CLIP [27] IN THE PRACTICE

- OpenAI model trained on 400M (image, text) pairs from the internet
- Texts contains semantic facts that describe the contents of the images
- It does not learn categories, learns how images matches descriptions

Example

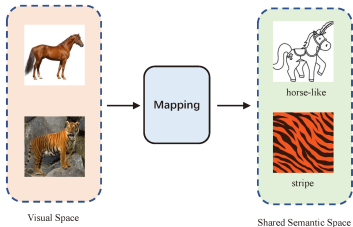


*a photo of a black and white
bird standing on ice*

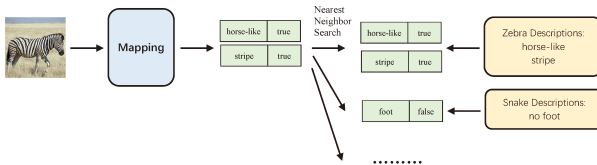
- black and white pattern
- ice
- bird shape

CLIP [27] IN THE PRACTICE

Training and inference example



(a) Training step



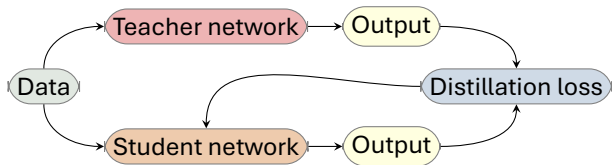
(b) Prediction step

Source

SELF-DISTILLATION WITH NO LABELS (DINO) [28]

Training method for Vision Transformers (or CNN)

- Student-teacher framework



- Self-supervised learning
 - The model learns without labeled data
 - Contrastive learning teaches the model to differentiate between similar and dissimilar pairs of data

SELF-DISTILLATION WITH NO LABELS (DINO) [28]

Training approach

1. DINO generates various crops from an image
2. Crops are passed through the student model
Global views are processed by the teacher model
 - Student learns to relate image patches within the global context
3. Minimize the loss function that measures the similarity between the student and teacher without using labeled data.
4. Update the weights of the Student by back-propagation
5. Update the weights of the Teacher from the Student weights using Exponential Moving Average (EMA)

SELF-DISTILLATION WITH NO LABELS (DINO) [28]

Usage

- Pretrain a ViT without labels

or

💡 Extract features using a pretrained DINO ViT

- Image classification
- Object retrieval
- Segmentation
- Clustering

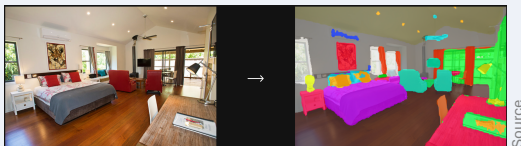
DINOv2 [29]

Approach

- Improves DINO training approach and gets stronger features
- Scalable to large ViTs with efficient training

Usage

- Download pretrained models in 142M curated web images¹⁴
- Use them as backbones to achieve competitive results in several tasks¹⁵



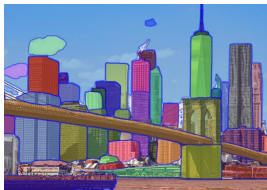
¹⁴<https://github.com/facebookresearch/dinov2>

¹⁵<https://dinov2.metademolab.com/>

SEGMENT ANYTHING MODEL (SAM) [30]

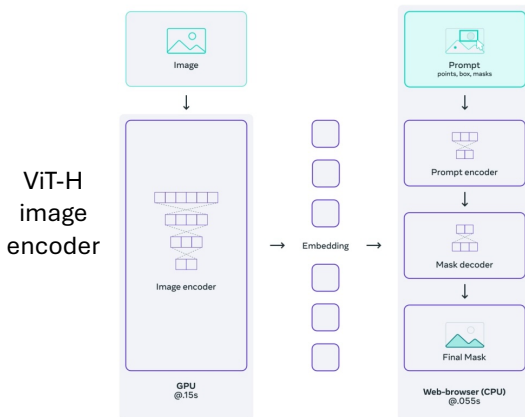
- Proposed in 2023 by Meta AI
- Predicts object masks only
- Trained on SA-1B dataset
 - 1 billion masks over 11 million images
- Zero-shot transfer
 - It adapts to new image distributions and tasks without prior knowledge

⚠ 636M parameters



Source

SAM [30]



Create an embedding
from input prompts
(clicks, bboxes)

Lightweight transformer
predicts object masks
from image and
prompt embeddings

SAM [30]

- Performance
 - $\approx 0.15\text{s}$ on a NVIDIA A100 GPU for image encoding
 - $\approx 50\text{ms}$ on CPU for prompt encoding and mask decoding
- Tasks
 - Segment an object from a single foreground point
 - Edge detection
 - Object proposal
 - Instance segmentation
 - Text-to-mask (using CLIP embeddings)

SAM EFFICIENT VARIANTS

MobileSAM [31]

- Use knowledge distillation to train a smaller image encoder from the original heavy image encoder
- Fine tune the decoder

FastSAM [32]

- CNN-based solution
- Two stages
 - All-instance segmentation (YOLO-V8)
 - Optional prompt-guided selection (CLIP)

Model	Parameters	ms/image (CPU)
Meta SAM-b	93.7	49401
Meta SAM-t	38.9	25997
MobileSAM	10.1	25381
FastSAM-s	11.8	55.9

Source



**THANK YOU
FOR YOUR TIME**

INDITEXUDC

Cátedra de IA en Algoritmos Verdes

<https://catedrainditexalgoritmosverdes.com/>

BIBLIOGRAPHY I

-  M. Elgendy, *Deep Learning for Vision Systems*. Manning, 2020.
-  F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “Squeezenet: Alexnet-level accuracy with 50x fewer parameters and <0.5mb model size,” 2016. [Online]. Available: <https://arxiv.org/abs/1602.07360>
-  S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” 2016. [Online]. Available: <https://arxiv.org/abs/1510.00149>
-  A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>

BIBLIOGRAPHY II

-  M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” 2019. [Online]. Available: <https://arxiv.org/abs/1801.04381>
-  A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, “Searching for mobilenetv3,” 2019. [Online]. Available: <https://arxiv.org/abs/1905.02244>
-  X. Zhang, X. Zhou, M. Lin, and J. Sun, “Shufflenet: An extremely efficient convolutional neural network for mobile devices,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.01083>
-  M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” 2020. [Online]. Available: <https://arxiv.org/abs/1905.11946>

BIBLIOGRAPHY III



S. Mehta and M. Rastegari, “Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer,” 2022. [Online]. Available: <https://arxiv.org/abs/2110.02178>



R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” 2014. [Online]. Available: <https://arxiv.org/abs/1311.2524>



R. Girshick, “Fast r-cnn,” 2015. [Online]. Available: <https://arxiv.org/abs/1504.08083>



S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” 2016. [Online]. Available: <https://arxiv.org/abs/1506.01497>

BIBLIOGRAPHY IV



W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, *SSD: Single Shot MultiBox Detector*. Springer International Publishing, 2016, p. 21–37. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-46448-0_2



J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2016. [Online]. Available: <https://arxiv.org/abs/1506.02640>



J. Redmon and A. Farhadi, “Yolo9000: Better, faster, stronger,” 2016. [Online]. Available: <https://arxiv.org/abs/1612.08242>



——, “Yolov3: An incremental improvement,” 2018. [Online]. Available: <https://arxiv.org/abs/1804.02767>



A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “Yolov4: Optimal speed and accuracy of object detection,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.10934>

BIBLIOGRAPHY V



C. Li, L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie, Y. Li, B. Zhang, Y. Liang, L. Zhou, X. Xu, X. Chu, X. Wei, and X. Wei, “Yolov6: A single-stage object detection framework for industrial applications,” 2022. [Online]. Available: <https://arxiv.org/abs/2209.02976>



C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” 2022. [Online]. Available: <https://arxiv.org/abs/2207.02696>



M. Tan, R. Pang, and Q. V. Le, “Efficientdet: Scalable and efficient object detection,” 2020. [Online]. Available: <https://arxiv.org/abs/1911.09070>

BIBLIOGRAPHY VI



N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” 2020. [Online]. Available: <https://arxiv.org/abs/2005.12872>



Z. Yao, J. Ai, B. Li, and C. Zhang, “Efficient detr: Improving end-to-end object detector with dense prior,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.01318>



O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” 2015. [Online]. Available: <https://arxiv.org/abs/1505.04597>



C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, and N. Sang, “Bisenet v2: Bilateral network with guided aggregation for real-time semantic segmentation,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.02147>

BIBLIOGRAPHY VII

-  J. Xu, Z. Xiong, and S. P. Bhattacharyya, “Pidnet: A real-time semantic segmentation network inspired by pid controllers,” 2023. [Online]. Available: <https://arxiv.org/abs/2206.02066>
-  D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, “Yolact: Real-time instance segmentation,” 2019. [Online]. Available: <https://arxiv.org/abs/1904.02689>
-  A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>

BIBLIOGRAPHY VIII



M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.14294>



M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, “Dinov2: Learning robust visual features without supervision,” 2024. [Online]. Available: <https://arxiv.org/abs/2304.07193>

BIBLIOGRAPHY IX



A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, “Segment anything,” 2023. [Online]. Available: <https://arxiv.org/abs/2304.02643>



C. Zhang, D. Han, Y. Qiao, J. U. Kim, S.-H. Bae, S. Lee, and C. S. Hong, “Faster segment anything: Towards lightweight sam for mobile applications,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.14289>



X. Zhao, W. Ding, Y. An, Y. Du, T. Yu, M. Li, M. Tang, and J. Wang, “Fast segment anything,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.12156>